

Шар оюн автоматы

Машина IOI катышуучулардын көңүлүн ачуу үчүн шар оюн автоматын ойлоп тапкан. Автоматтын ички түзүлүшү төмөнкүдөй:

- Автомат 0 дан $N - 1$ чейин нумурланган N **түйүндөрүнөн** турат. $N - 1$ түйүнү **тамыр** деп аталат.
- $0 \leq u < N - 1$ аткарылган ар бир u түйүнү үчүн, u түйүнүнүн **ата-энеси** $P[u]$ түйүнү болуп саналат, жана $P[u] > u$ аткарылат. Ал эми, u түйүнү $P[u]$ түйүнүнүн **баласы** болуп саналат. Тамырдын ата-энеси жок. Балдары жок түйүн **жалбырак** деп аталат.

Сиз N 'ди же ар кандай u түйүнүнүн ата-энеси $P[u]$ 'лерды **билбейсиз**. Анын ордуна, Машина сизге M – жалбырактардын санын – жана жалбырактардын нумурларды $0, 1, \dots, M - 1$ экенин билдирди. Сиздин тапшырмаңыз – автоматты иштетүү аркылуу анын ички түзүлүшүн аныктоо.

Автоматтын ар бир түйүнү эң көп дегенде бир **шарды** батыра алат жана ар бир шардын терс эмес бүтүн сан болгон **мааниси** бар. Эгерде түйүн x маанисине ээ болгон шарды камтыса, ал түйүн x маанисине ээ деп айтабыз. Эгерде түйүндүн ичинде шар болсо, ал **ээленген** болот; болбосо, ал **бош** болот. Башында ар бир түйүн бош болот.

Сиз эки түрдөгү операцияларды аткара аласыз:

- **insert**(U, X): X маанисин жаңы шарга жазып, аны U жалбырака киргизүүгө аракет кылуу.
 - Эгерде U жалбырагы ээленген болсо, операция шарды киргизбестен `false` кайтарат.
 - Эгерде U жалбырагы бош болсо, шар U түйүнүнө жайгаштырылат. Андан кийин шар, ата-эне бош турганда, учурдагы түйүнүнүн ата-энесине жылат берет. Шар тамырга же ата-энеси ээленген түйүнгө жеткенде кыймыл токтойт. Операция `true`'ну кайтарат.
- **collect**(): эгерде тамыр бош болсо (бул автомат бош дегенди билдирет) жана `collect` бош массив кайтарат. Болбосо, төмөндө сүрөттөлгөн `traverse` рекурсивдүү кыймыл аракет учурда автоматта турган бардык шарлардын маанилерин S массивине чогултат. Башында, S массиви бош болуп `traverse(N - 1)` чакырылат.

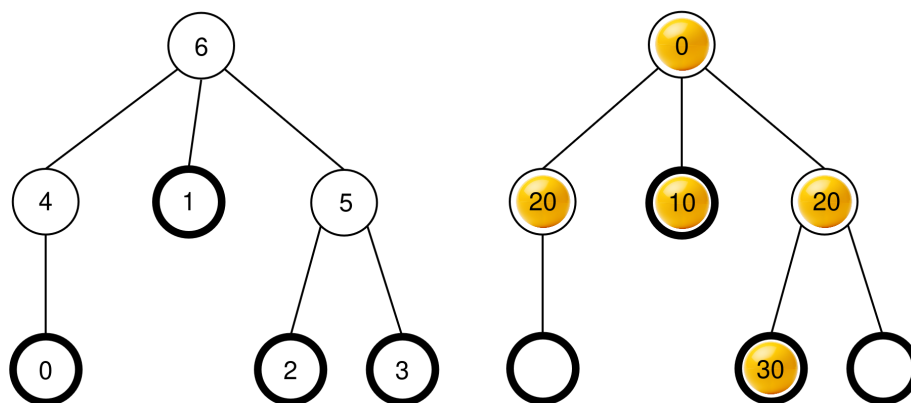
```

traverse(u) :
    u түйүнүнүн маанисин S'тин акырына салуу
    c[u] – u түйүнүнүн ээленген балдарынын тизмеси болсун
    c[u] тизмесин ошол түйүндөрдөгү шарлардын
    маанисине жараша өсүүчү тартипте сорттоо
    (эгер бир нече түйүндүн мааниси бирдей болсо,
    алар каалаган тартипте жайгашышы мүмкүн)
    c[u] тизмесиндеги ар бир v үчүн:
        traverse(v)

```

Бул процедура аяктагандан кийин, **автоматтагы бардык шарлар алып салынат**, жана `collect S'`ти кайтарат.

Мисалы, $N = 7$ түйүнү жана `parent` массиви $P = [4, 6, 5, 5, 6, 6]$ болгон автоматты карап көрүңүз. Сол жактагы сүрөт бош автоматтын түйүндөрдүн индекси менен көрсөтсө, оң жактагы сүрөт `insert` операцияларынын белгилүү бир катарынан кийин шарлардын мүмкүн болгон конфигурациясын көрсөтөт (бул катар Мисалдар бөлүмүндө берилет).



When `collect()` is called, the root (node 6) is occupied, so S is initialized as $S = []$ and `traverse(6)` is called.

`collect()` чакырылганда, тамыр (6) ээленгендиктен, $S = []$ менен `traverse(6)` чакырылат.

traverse(6) : 6 түйүнү 0 маанисине ээ; аны S катарына кошот $S = [0]$. 6 түйүнүнүн балдары 4, 1 жана 5 болуп, алардын баары ээленген. Алардын маанилери тиешелүү түрдө 20, 10 жана 20. Азайбаган тартипте иреттесе $c[6] = [1, 5, 4]$ болот ($c[6] = [1, 4, 5]$ да жарактуу болорун эске алыңыз, анткени 4 жана 5 түйүндөрү бирдей мааниге ээ). Андан кийин кыймыл аракет $c[6]$ иреттеги ар бир түйүн үчүн `traverse` ошол тартипте чакырат.

- **traverse(1) :** 1 түйүнүнүн мааниси 10; аны S түйүнүнө кошуу $S = [0, 10]$ берет. 1 түйүнүнүн ээленген балдары жок, ошондуктан бул чакыруу аяктайт.
- **traverse(5) :** 5 түйүнү 20 маанисине ээ; аны S түйүнүнө кошуу $S = [0, 10, 20]$ маанисин берет. 5 түйүнүндө бир ээленген бала бар, 2 түйүнү, ошондуктан $c[5] = [2]$

жана процедура `traverse(2)` чакырат.

- **`traverse(2)`** : 2 түйүнүнүн мааниси 30 ; аны S түйүнүнө кошуу $S = [0, 10, 20, 30]$ берет. 2 түйүнүнүн ээленген балдары жок, ошондуктан бул чакыруу аяктайт.
- Аткаруу `traverse(5)` функциясына кайтып келет, ал $c[5]$ ичиндеги бардык балдарды карап чыкты, ошондуктан анын чакыруусу аяктайт.
- **`traverse(4)`** : 4 түйүнүнүн мааниси 20 ; аны S түйүнүнө кошуу $S = [0, 10, 20, 30, 20]$ берет. 4 түйүнүнүн ээленген балдары жок, ошондуктан бул чакыруу аяктайт.

Бардык чакыруулар аяктады жана текшерүү үчүн башка түйүндөр жок. Акырында, ар бир шар автоматтан алынып салынат жана `collect` $S = [0, 10, 20, 30, 20]$ массивин кайтарат.

Сиздин тапшырмаңыз N түйүндөрүнүн санын аныктоо жана автоматтын түзүлүшүн сүрөттөгөн, бирок жалбырак жана тамыр эмес түйүндөрдүн башкача аталышы мүмкүн болгон $R = [R[0], R[1], \dots, R[N-2]]$ ата-энелер массивин билдирүү. Формалдуу түрдө, автоматтын ар бир u түйүнүнө $0 \leq L[u] < N$ менен уникалдуу $L[u]$ белгисин дайындоо мүмкүн болсо жана төмөнкү шарттар аткарылса, билдирилген R массиви туура деп эсептелет:

- $L[u] = u$ бардык $0 \leq u < M$ жана $u = N - 1$ үчүн, жана
- $L[P[u]] = R[L[u]]$ бардык $0 \leq u < N - 1$ үчүн.

$R[u] > u$ болушу милдеттүү эмес. Сиз чечимиңизди кайтарганда автомат **бош** болушу керек.

K – аткарылган `collect` операцияларынын саны, ал эми B бардык `insert` чакыруулары арасында шарга жазылган максималдуу маани болсун. $C = K + B$ болсун. Анда C **мааниси 1000'дөн ашпашы керек**. Айрым кошумча тапшырмалардагы упайыңыз C маанисине жараша болот.

Ишке ашыруунун чөө-жайы

Сиз төмөнкү жол-жобону ишке ашырышыңыз керек.

```
std::vector<int> find_structure(int M)
```

- M : автоматтагы жалбырактардын саны.
- Процедура ар бир тест учуру үчүн бир гана жолу чакырылат.
- Процедура узундугу $N - 1$ болгон $R = [R[0], R[1], \dots, R[N - 2]]$ – туура ата-эне массивди кайтарышы керек.

Автомат менен өз ара аракеттенүү үчүн, сиздин жол-жобоңуз төмөнкү эки процедураны чакыра алат.

Биринчи жол-жобо төмөнкүдөй:

```
bool insert(int U, int X)
```

- U : шар киргизилиши керек болгон жалбырактын индекси. $0 \leq U < M$ шарты аткарылышы керек.
- X : шардын бүтүн сан мааниси. $0 \leq X \leq 1000$ шарты аткарылышы керек.
- Бул процедура эгерде шар ийгиликтүү киргизилген болсо, `true` кайтарат, ал эми U жалбырагы мурунтан эле ээленген болсо, `false` кайтарат.
- Бул процедураны эң көп дегенде 500 000 жолу чакырууга болот.

Экинчи жол-жобо төмөнкүдөй:

```
std::vector<int> collect()
```

- Бардык киргизилген шарладын маанилерин чогултат, автоматты бошотот жана чогултулган S массивин кайтарат.

Эгерде программаңыздын аткарылышынын кайсы бир учурунда S мааниси 1000'дөн ашып кетсе, анда сиздин чечимиңиз `Output isn't correct: Too many resources used` деген чечимди алат.

Автоматтын түзүлүшү `find_structure` чакырылганга чейин бекитилет. Грейдер **детерминисттик**, башкача айтканда, эгер сиз аны эки жолу иштетсеңиз жана эки жолу тең бирдей операциялардын ырааттуулугун аткарасыз, `collect` функциясына чакыруулар бирдей массивдерди кайтарат.

The grader is **deterministic** in the sense that if you run it twice and both runs perform the same sequence of operations, the calls to `collect` will return the same arrays.

Чектөөлөр

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Кошумка Тапшырмалар

Кошумча тапшырмалар	Упай	Кошумча чектөөлөр
1	5	Тамырдын так M баласы бар.
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	Кошумча чектөөлөр жок.

3 жана 4-кошумча тапшырмаларда сиздин упайыңыз төмөнкүчө C маанисине көз каранды.

3-кошумча тапшырма (25 упай)

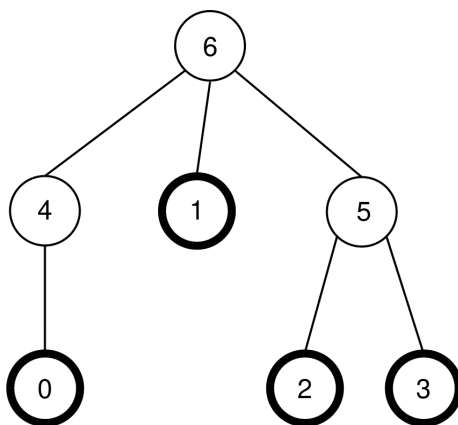
Чектөөлөр	Упай
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

4-кошумча тапшырма (60 упай)

Чектөөлөр	Упай
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Мисал

Тапшырманын сүрөттөмөсүндөгүдөй эле автоматты карап көрөлү, анда, $N = 7$, $M = 4$ жана $P = [4, 6, 5, 5, 6, 6]$. Автомат кайрадан төмөнкү сүрөттө көрсөтүлгөн:



Грейдер төмөнкү чакырууларды жасайт:

```
find_structure(4)
```

Процедура төмөнкү чакырыктардын ырааттуулугун жасай алат:

1. `insert(0, 0)` : 0 жалбырагы бош, ошондуктан 0 мааниси бар шар 0 жалбырагына жайгаштырылат. $P[0] = 4$ түйүнү бош, ошондуктан шар 4 түйүнүнө жылат. $P[4] = 6$ түйүнү бош, ошондуктан шар 6 түйүнүнө жылат. 6 түйүнү тамыр болгондуктан, кыймыл токтойт. Чакыруу `true` кайтарат.
2. `insert(3, 20)` : 3 жалбырагы бош, ошондуктан 20 мааниси бар шар 3 жалбырагына жайгаштырылат. $P[3] = 5$ түйүнү бош, ошондуктан шар 5 түйүнүнө жылат. $P[5] = 6$ ээленип тургандыктан, кыймыл токтойт. Чакыруу `true` кайтарат.
3. `insert(1, 10)` : 1 жалбырагы бош, ошондуктан 10 маанисиндеги шар 1 жалбырагына жайгаштырылат. $P[1] = 6$ ээленип болгондуктан, шар 1 түйүнүндө калат. Чакыруу `true` кайтарат.
4. `insert(2, 30)` : 2 жалбырагы бош, ошондуктан 30 маанисиндеги шар 2 жалбырагына жайгаштырылат. $P[2] = 5$ ээленип болгондуктан, шар 2 түйүнүндө калат. Чакыруу `true` кайтарат.
5. `insert(1, 25)` : 1 жалбырагы ээленген, ошондуктан шар 1 жалбырагына коюлган эмес. Чакыруу `false` кайтарат.
6. `insert(0, 20)` : 0 жалбырагы бош, ошондуктан 20 мааниси бар шар 0 жалбырагына жайгаштырылат. $P[0] = 4$ түйүнү бош, ошондуктан шар 4 түйүнүнө жылат. $P[4] = 6$ ээленип тургандыктан, кыймыл токтойт. Чакыруу `true` кайтарат.

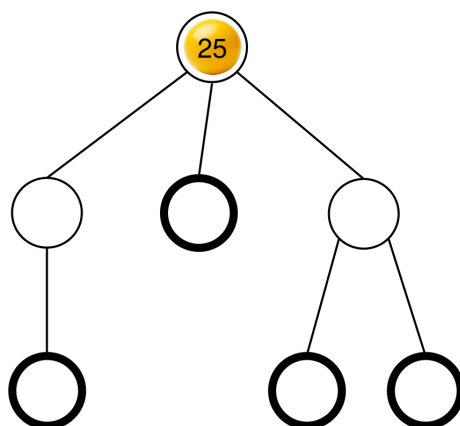
Натыйжадагы конфигурация тапшырманын сүрөттөмөсүндө көрсөтүлгөн.

Андан кийин, процедура төмөнкүлөрдү чакыра алат:

```
collect()
```

Бул чакыруунун аткарылышы тапшырманын сүрөттөмөсүндө түшүндүрүлгөн, ошондуктан бул чакыруу $S = [0, 10, 20, 30, 20]$ кайтарат. Андан кийин, автомат кайрадан бош болот.

Андан кийин, процедура `insert(2, 25)` 'ны чакыра алат. 2 жалбырагы бош, ошондуктан 25 маанисиндеги шар 2 жалбырагына жайгаштырылат. Андан кийин бул шар 5 түйүнүнө, андан кийин 6 түйүнүнө өтөт, ал жерде токтойт. Чакыруу `true` кайтарат. Натыйжадагы конфигурациясы төмөнкү сүрөттө көрсөтүлгөн.



Акырында, процедура `collect()` функциясын чакыра алат, ал $S = [25]$ кайтарып, автоматты бошотот.

`find_structure` кайтара ала турган эки туура ата-эне массиви бар: $R = P = [4, 6, 5, 5, 6, 6]$ (бул $L = [0, 1, 2, 3, 4, 5, 6]$ белгилөөсүнө туура келет), жана $R = [5, 6, 4, 4, 6, 6]$ (бул $L = [0, 1, 2, 3, 5, 4, 6]$ белгилөөсүнө туура келет). Бул мисалда, $K = 2$ жана $B = 30$, ошондуктан $C = 32$.

Үлгү грейдер

Киргизүү форматы:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Чыгаруу форматы:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Бул жерде, Q – `find_structure` кайтарган R массивинин узундугу.